LifeOS Agent: Project Proposal

Nov 1, 2026 · @Shanaya Mahendran

Overview

LifeOS Agent is a Next.js web app where one chat message updates a built-in calendar that organizes career, studies, competitions, health and growth in one place.

The problem. Exams, hackathon deadlines, volunteering shifts, open source work, karate, sleep and blogging all compete for the same hours. Today they live in separate apps and notes, so conflicts show up late and long-term goals get squeezed out by urgent tasks.

The solution. An orchestrator agent reads each chat message, hands it to the right domain agent (Career, Education, Competitions and so on), checks for conflicts, and writes the result to one master calendar. Every domain keeps its own instance, with its own goals, memory and calendar layer, so each area can be viewed and managed separately or together.

Goals and non-goals

The MVP succeeds if a week of real life can be planned and adjusted entirely through chat, without opening another planner.

Goals

- Create, move and cancel events with plain-language chat messages
- One master calendar with a color-coded layer per life domain
- Automatic conflict detection with suggested alternatives
- Protected time for sleep, meals and karate that the agent defends
- A daily morning brief and a weekly review
- Separate agent instances per domain, each with its own goals and memory

Non-goals (first version)

- Multi-user or team scheduling
- A native mobile app (the web app is built mobile-first instead)
- Deep integrations with every external tool; Google Calendar sync comes later

Life domains

The agent starts with 12 domains (your 11 plus rest and buffer time), and 6 more are suggested below. Each domain is its own agent instance and calendar layer.

Domain	What the agent manages
Career	Applications, CV and LinkedIn upkeep, networking, interview prep
Education	Lectures, assignments, exam countdowns, revision blocks
Competitions	Hackathon deadlines, team tasks, pitch rehearsals, results
Volunteering	Event shifts, hours logged, commitments
Open source	Issues picked, PR status, weekly contribution slots
Food	Meal windows, prep time, simple meal plan
Sleep	Bedtime, wake time, wind-down reminders
Karate	Training sessions, conditioning, grading dates
Blogging	Idea to draft to publish pipeline, writing slots
Learning	Skill roadmap, daily learning blocks, project practice
Forward planning	Weekly review, monthly goals, yearly direction
Rest and buffer	Protected downtime and slack between tasks

Suggested additions (not on your list)

- **Finance:** budget, scholarship and prize tracking
- **Admin:** forms, fees, applications, renewals
- **Social and mentors:** family, friends, mentor check-ins
- **Portfolio:** projects, certificates, achievements log
- **Travel and commute:** buffer time between places
- **Habits and reflection:** daily check-in, streaks, mood and energy notes

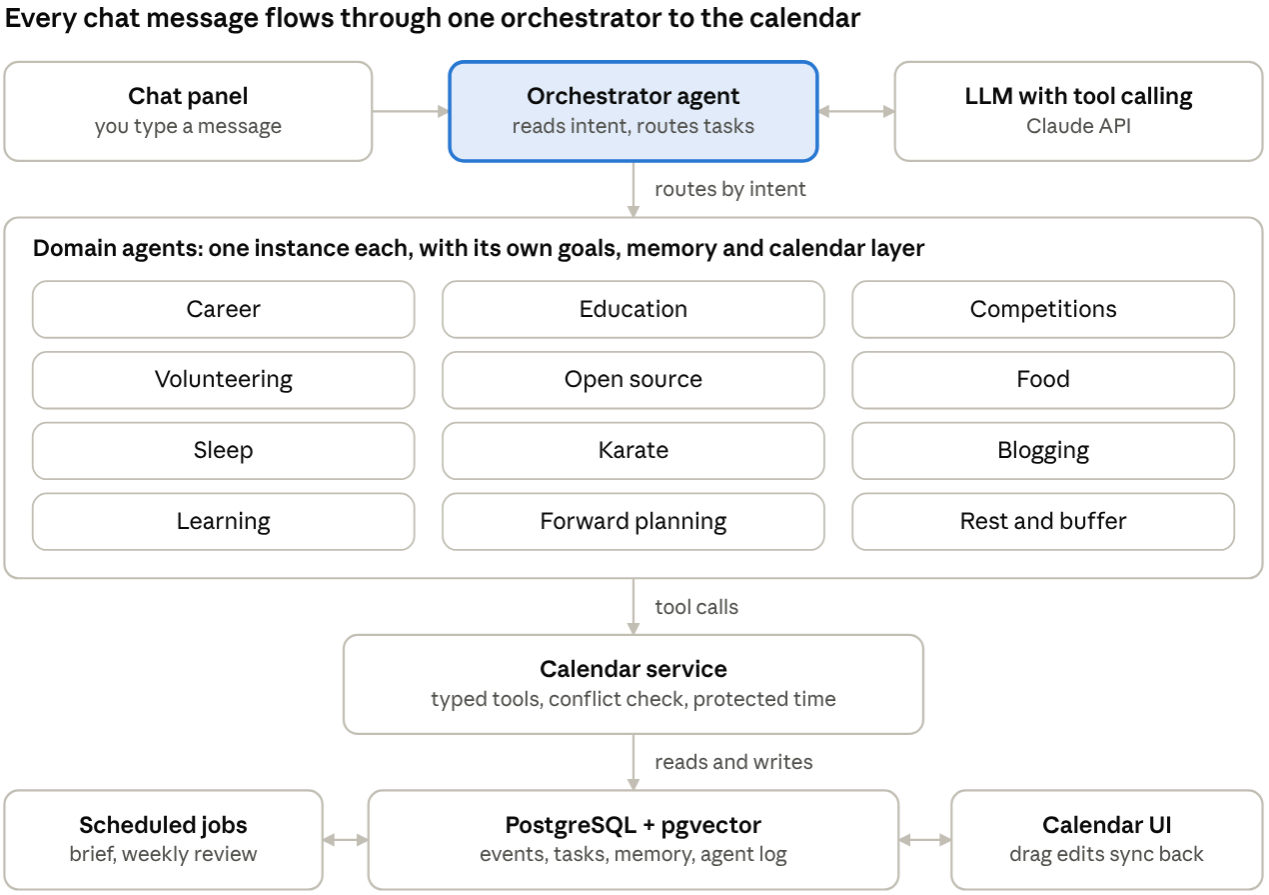
Core features

Six features make the product: chat control, a built-in calendar, domain instances, conflict handling, auto-scheduling and rhythm reviews.

1. **Chat-to-calendar.** Messages like "Move karate to 7 PM" or "Block 2 hours for the pitch tomorrow" create, move or cancel events. The agent replies with what changed.
2. **Built-in calendar.** Month, week and day views with drag-and-drop editing. Edits made by hand sync back to the agent's memory.
3. **Domain instances.** Each domain has its own goals, notes, task list and calendar color. Layers can be toggled on or off.
4. **Conflict detection.** Overlaps and overload are flagged before saving, with two or three alternative slots offered.
5. **Auto-scheduling.** Deadlines generate work blocks. An exam in 10 days produces revision sessions; a hackathon submission produces build and pitch-prep slots.
6. **Briefs and reviews.** A morning brief lists the day's plan. A Sunday review shows what was done, what slipped, and proposes next week.

Protected time. Sleep, meals and karate are marked as protected. The agent warns before overbooking them and asks for explicit confirmation.

System architecture



architecture · 1 orchestrator, 12 domain agents, 1 database

A chat message reaches the orchestrator, which asks the LLM to interpret it and routes the task to a domain agent. Every write goes through one calendar service, so conflicts and protected time are checked in a single place, and the calendar UI reads the same database.

Tech stack

The whole product is one Next.js (App Router, TypeScript) codebase, with the AI layer behind server-side route handlers.

Layer	Choice	Why
Frontend	Next.js, React, Tailwind CSS	One codebase for UI and API, mobile-first layout
Calendar UI	FullCalendar (or react-big-calendar)	Month, week, day views, drag-and-drop out of the box

Layer	Choice	Why
Backend	Next.js route handlers and server actions	No separate server to run
Database	PostgreSQL with Prisma	Relational fit for events, domains and tasks
AI layer	LLM with tool calling (Claude API)	Agent calls typed calendar functions
Agent memory	pgvector in the same Postgres	Per-domain notes and past decisions, no extra service
Auth	Auth.js (NextAuth)	Simple login, Google sign-in ready
Notifications	Web push and email reminders	Morning brief, deadline and bedtime alerts
Scheduled jobs	Vercel Cron or a queue worker	Daily brief, weekly review, auto-scheduling
Hosting	Vercel and a managed Postgres (Neon or Supabase)	Free tiers cover an MVP

Data model

Six tables carry the system; every event belongs to exactly one domain, which is what makes per-domain layers and instances possible.

Table	Key fields
User	id, name, email, timezone, sleep window, preferences
Domain	id, user_id, name, color, goals, is_protected, agent_config
Event	id, domain_id, title, start, end, recurrence, status, source (chat or manual), priority
Task	id, domain_id, title, due date, estimate (minutes), status, linked_event_id
Memory	id, domain_id, note, embedding, created_at
AgentLog	id, message, parsed_intent, tool_calls, result, created_at

AgentLog stores every chat command with the tool calls it triggered, so any change can be audited and undone.

Agent design

The agent never edits the database directly; it can only call a small set of typed tools, and every write is checked for conflicts first.

Tools the agent can call

Tool	Purpose
create_event	Add an event to a domain layer
move_event	Change an event's time
cancel_event	Remove or mark an event cancelled
query_schedule	Read events for a date range or domain
find_free_slots	Suggest open times that respect protected blocks
create_task	Add a task with a due date and time estimate
plan_backwards	Turn a deadline into work blocks
save_memory	Store a note in a domain's memory

Example chat commands

You type	The agent does
"Move karate to 7 PM today"	Finds the Karate event, checks 7 PM is free, moves it
"Hackathon pitch is due Friday"	Creates the deadline, adds prep blocks on earlier days
"What does my week look like?"	Reads all layers and summarizes by day
"I'm tired, lighten tomorrow"	Proposes which low-priority blocks to drop, waits for approval

Safety rules

- Moves that touch protected time (sleep, meals, karate) need a confirmation reply
- Bulk changes of 5 or more events are previewed before saving
- Every change is logged and can be undone with "undo"

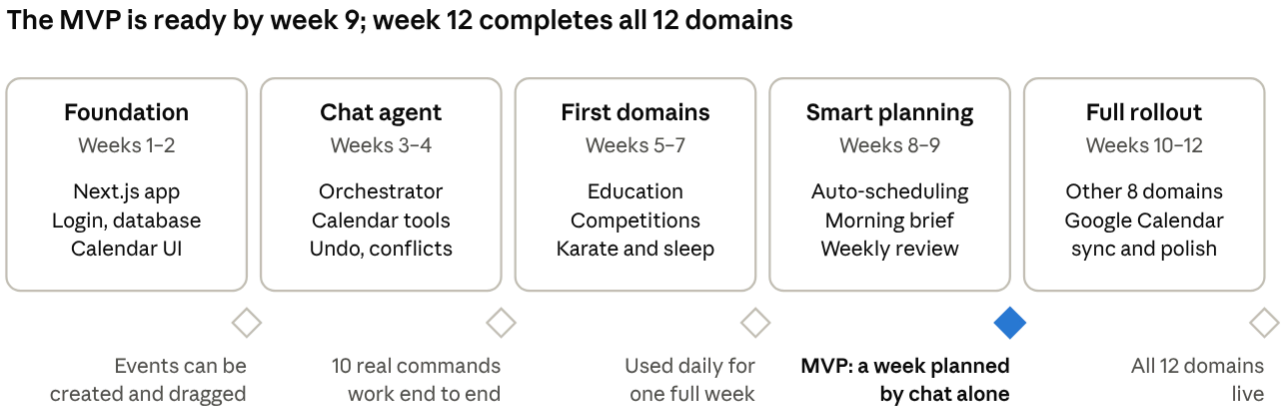
UI and UX

The main screen is a calendar with a chat panel beside it (a bottom sheet on phones), so every command and its result are visible together.

- **Home (Today):** morning brief, today's timeline, next three tasks, a sleep and karate status strip
- **Calendar:** month, week and day views; domain layers toggled by color chips; drag to move, click to edit
- **Chat panel:** always available; shows what the agent changed with an undo button on each reply
- **Domain pages:** one per domain with goals, task list, notes and its own mini calendar
- **Weekly review:** planned versus done per domain, slipped items, and a proposed next week
- **Settings:** sleep window, protected blocks, working hours, notification times

Design principles: mobile-first, dark mode, one primary action per screen, and no more than two taps to reach any domain.

Roadmap



roadmap · 5 phases, 5 gates

Each gate is a working checkpoint, and the next phase starts only once the previous one passes it. Timings are rough estimates and can shift around exams and competitions.

Risks and success metrics

The biggest risk is over-scheduling: a planner that packs every hour gets abandoned, so buffer time is a first-class feature.

Risk	Mitigation
Agent misreads a command and edits the wrong event	Typed tools, confirmation on protected or bulk changes, undo on every change
Over-scheduled weeks lead to burnout	Mandatory buffer blocks and a weekly load warning
LLM cost and latency	Small model for intent parsing, larger model only for planning and reviews
Scope creep across 12 domains	MVP ships 4 domains first, others reuse the same template
Personal data privacy	Row-level security, no data sold or shared, export and delete options

Success metrics

- 80% of calendar edits made through chat after the first two weeks
- Under 5 seconds from message to calendar update
- Zero missed deadlines in the MVP domains over one month of real use

- Sleep and karate blocks kept in at least 90% of weeks